Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №3
з дисципліни «Технології та методика програмування вбудованих систем»

«Моделювання модулю універсального асинхронного приймача-передавача сім'ї AVR»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
ст. викладач, к.т.н Орленко Сергій Петрович

Дата здачі: 28.03.2026

КИЇВ – 2026

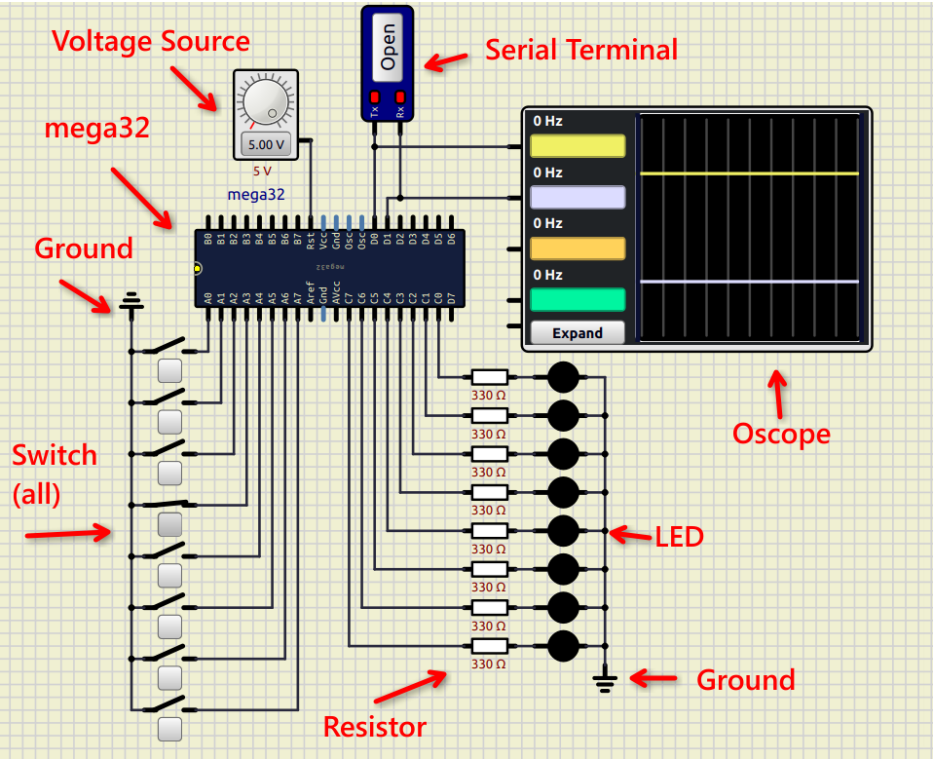
Тема. Моделювання модуля універсального асинхронного приймача-передавача сім'ї AVR.

Мета. Користуючись пакетом SimulIDE дослідити моделювання модулю універсального асинхронного приймача-передавача сім'ї AVR.

Завдання. Першим кроком необхідно створити модель пристрою в пакеті SimulIDE. Далі потрібно розробити схему алгоритму роботи моделі та робочу програму. Після цього слід створити hex-файл та підключити його до мікроконтролера. Наступним етапом є запуск моделі та виконання її дослідження згідно методичних вказівок. Наприкінці необхідно зробити відповідні висновки.

Хід роботи

Виконання лабораторної роботи розпочалося зі збирання схеми в середовищі SimulIDE.



Оскільки в бібліотеці компонентів відсутня модель мікроконтролера ATmega8515, замість неї було використано мікроконтролер ATmega32. Це рішення є оптимальним, оскільки ATmega32 має абсолютно ідентичну архітектуру конфігураційних регістрів для роботи з модулем УАПП, а також необхідні для підключення периферії порти A та C. В налаштуваннях мікроконтролера частоту було змінено на 8 MHz, щоб забезпечити коректний розрахунок швидкості передачі даних у 9600 бод. Пін скидання RST був підключений до джерела живлення 5В (Voltage Source) для запобігання постійному перезавантаженню пристрою.

Для введення даних до пінів порту A були підключені 8 перемикачів. Було обрано саме фіксовані перемикачі (Switch), а не тактові кнопки, щоб зручно задавати та утримувати стабільний двійковий код під час зчитування порту. Зовнішні підтягуючі резистори для кнопок не використовувалися, оскільки в розробленій програмі здійснюється програмне увімкнення внутрішніх підтягуючих резисторів мікроконтролера. Для візуалізації отриманих даних до пінів порту C були підключені світлодіоди. Підключення здійснювалося строго послідовно: катоди світлодіодів заземлювалися, а аноди з'єднувалися з портом мікроконтролера через обмежувальні резистори номіналом 330 Ом, щоб уникнути пошкодження портів.

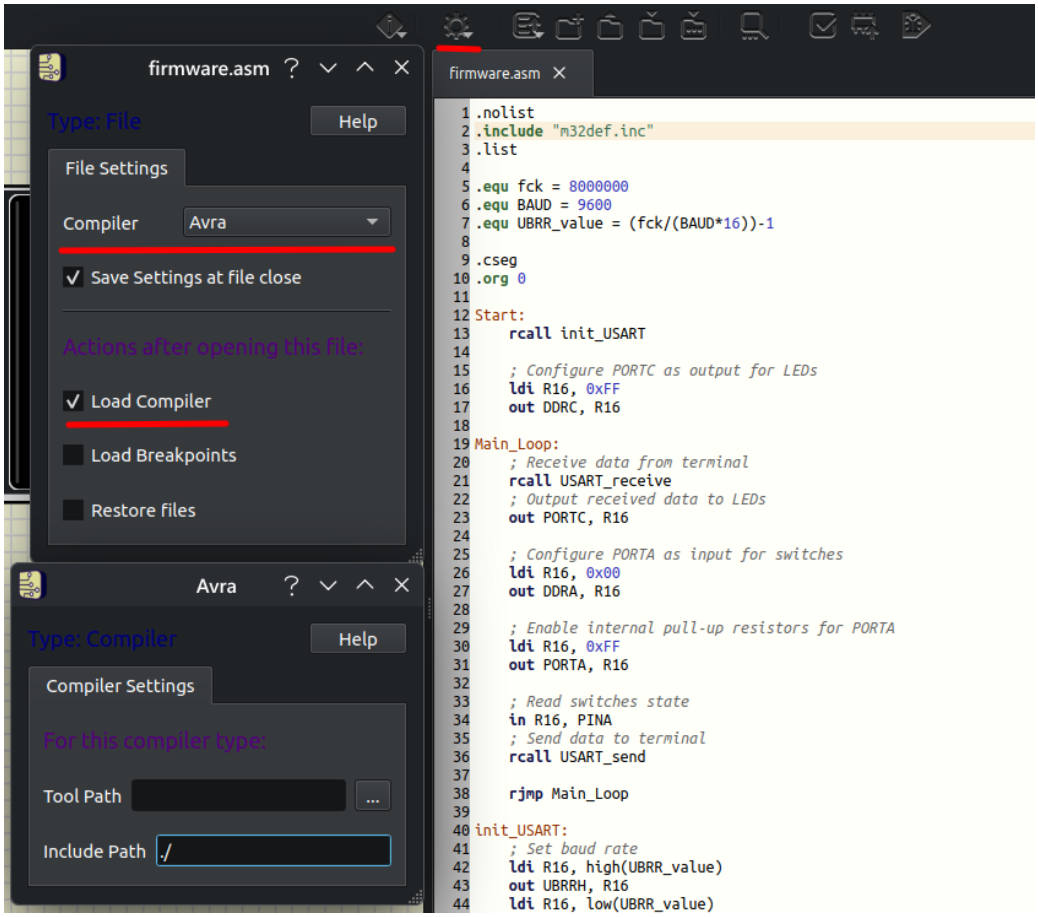
Для організації обміну даними було додано компонент віртуального термінала. З'єднання ліній зв'язку було виконано навхрест: контакт TX термінала підключено до піна D0 (RXD мікроконтролера), а контакт RX термінала – до піна D1 (TXD мікроконтролера).

Окремим важливим етапом налаштування середовища моделювання став вибір компілятора. Замість набору `avr-gcc` для збірки робочої програми було обрано асемблер `avra`. Компілятор `avr-gcc` орієнтований насамперед на мови високого рівня, а його вбудований асемблер використовує специфічний GNU-синтаксис, який суттєво відрізняється набором макросів та директив. Натомість алгоритм обміну даними по інтерфейсу УАПП розроблявся з використанням асемблера Atmel.

Використання `avra` дозволило зберегти оригінальну логіку та структуру коду без необхідності повного переписування інструкцій доступу до пам'яті та регістрів. Цей асемблер повністю підтримує стандартні директиви виділення сегментів та коректно обробляє файли конфігурації мікроконтролерів (зокрема `m32def.inc`), та чудово інтегрується у робоче середовище дистрибутивів на базі Linux, таких як Fedora.

```
xairaven@laptop ~
$ sudo dnf install avra
Updating and loading repositories:
Repositories loaded.
Package                                  Arch       Version                                  Repository                Size
Installing:
 avra                                    x86_64     1.4.2-10.fc43                            fedora                  1.8 MiB

Transaction Summary:
 Installing:         1 package

Total size of inbound packages is 151 KiB. Need to download 151 KiB.
After this operation, 2 MiB extra will be used (install 2 MiB, remove 0 B).
Is this ok [y/N]: y
[1/1] avra-0:1.4.2-10.fc43.x86_64                                          100% |   1.5 MiB/s | 151.1 KiB |  00m00s
--------------------------------------------------------------------------------------------------------------------------
[1/1] Total                                                                100% | 265.1 KiB/s | 151.1 KiB |  00m01s
Running transaction
[1/3] Verify package files                                                 100% | 500.0   B/s |   1.0   B |  00m00s
[2/3] Prepare transaction                                                  100% |   1.0   B/s |   1.0   B |  00m01s
[3/3] Installing avra-0:1.4.2-10.fc43.x86_64                               100% |   3.8 MiB/s |   1.9 MiB |  00m00s
Complete!
xairaven@laptop ~
$
```

Наступним етапом стало написання асемблерного коду програми. Компілятор був налаштований для використання в середовищі SimulIDE.



В оригінальний алгоритм було внесено кілька важливих адаптацій. По-перше, на початку програми додано ініціалізацію вказівника стеку (регістри SPH та SPL), що є критично необхідним для коректної роботи команд виклику підпрограм і запобігання критичним помилкам мікроконтролера. По-друге, налаштування кадру в регістрі UCSRC було змінено на формат без перевірки парності (8 біт даних, 1 стоп-біт), щоб синхронізувати його зі стандартними налаштуваннями термінала SimulIDE.

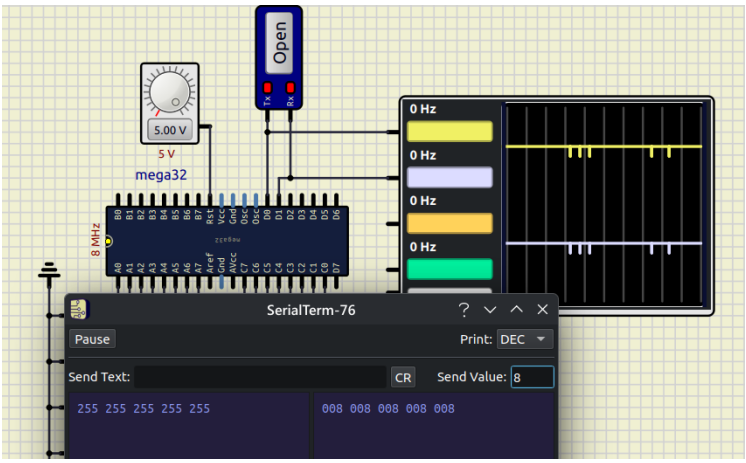
```
.nolist
.include "m32def.inc"
.list

.equ fck = 8000000
.equ BAUD = 9600
.equ UBRR_value = (fck/(BAUD*16))-1

.cseg
.org 0

Start:
; Initialize Stack Pointer
ldi R16, high(RAMEND)
out SPH, R16
```

```
ldi R16, low(RAMEND)
out SPL, R16

rcall init_USART

; Configure PORTC as output for LEDs
ldi R16, 0xFF
out DDRC, R16

Main_Loop:
; Receive data from terminal
rcall USART_receive
; Output received data to LEDs
out PORTC, R16

; Configure PORTA as input for switches
ldi R16, 0x00
out DDRA, R16

; Enable internal pull-up resistors for PORTA
ldi R16, 0xFF
out PORTA, R16

; Read switches state
in R16, PINA
; Send data to terminal
rcall USART_send

rjmp Main_Loop

init_USART:
; Set baud rate
ldi R16, high(UBRR_value)
out UBRRH, R16
ldi R16, low(UBRR_value)
out UBRRL, R16

; Enable receiver and transmitter
ldi R16, (1<<RXEN)|(1<<TXEN)|(0<<RXCIE)|(0<<TXCIE)|(0<<UDRIE)
out UCSRB, R16

; Set frame format: 8 data bits, no parity, 1 stop bit
ldi R16, (1<<URSEL)|(1<<UCSZ1)|(1<<UCSZ0)
out UCSRC, R16
ret

USART_send:
; Wait for empty transmit buffer
sending:
sbis UCSRA, UDRE
rjmp sending
; Put data into buffer, sends the data
out UDR, R16
ret

USART_receive:
; Wait for data to be received
receiving:
sbis UCSRA, RXC
rjmp receiving
; Get and return received data from buffer
in R16, UDR
ret
```

Процес компіляції пройшов успішно, прошивка була завантажена в мікросхему.

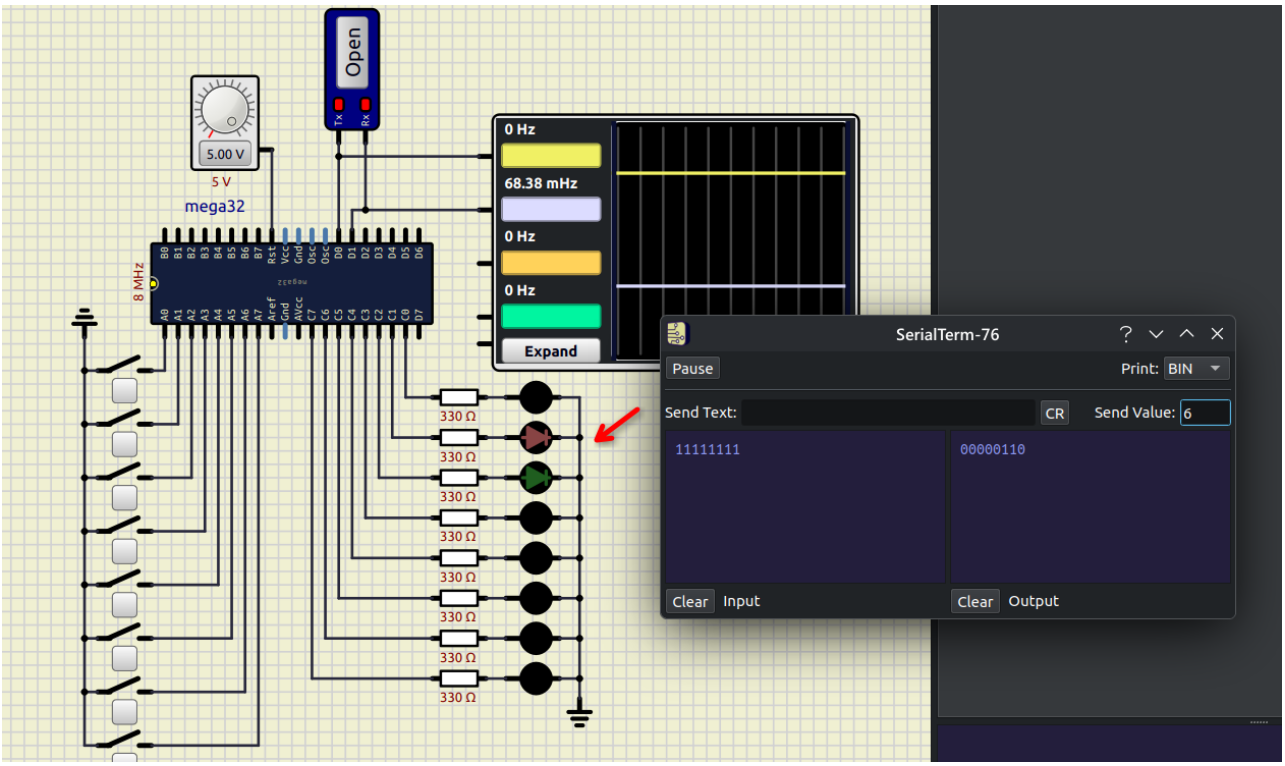
```
-----------------------------------------------------
Executing:
"avra" -W NoRegDef -l "/home/xairaven/Files/University/10/Embedded/Lab3/
code/"firmware.lst -I "./" "/home/xairaven/Files/University/10/Embedded/
Lab3/code/firmware.asm"

AVRA: advanced AVR macro assembler (version 1.4.2)
Pass 1...
Pass 2...
done


Assembly complete with no errors.
Segment usage:
   Code      :        29 words (58 bytes)
   Data      :         0 bytes
   EEPROM    :         0 bytes

/home/xairaven/Files/University/10/Embedded/Lab3/code//usr/share/avra/
m32def.inc(44) : PRAGMA PARTINC directive currently ignored
/home/xairaven/Files/University/10/Embedded/Lab3/code//usr/share/avra/
m32def.inc(48) : PRAGMA AVRPART directive currently ignored
/home/xairaven/Files/University/10/Embedded/Lab3/code//usr/share/avra/
m32def.inc(53) : PRAGMA AVRPART directive currently ignored
/home/xairaven/Files/University/10/Embedded/Lab3/code//usr/share/avra/
m32def.inc(760) : PRAGMA AVRPART directive currently ignored
/home/xairaven/Files/University/10/Embedded/Lab3/code//usr/share/avra/
m32def.inc(761) : PRAGMA AVRPART directive currently ignored
/home/xairaven/Files/University/10/Embedded/Lab3/code//usr/share/avra/
m32def.inc(762) : PRAGMA AVRPART directive currently ignored
/home/xairaven/Files/University/10/Embedded/Lab3/code//usr/share/avra/
m32def.inc(763) : PRAGMA AVRPART directive currently ignored
```

Під час тестування передачі даних за допомогою осцилографа було досліджено вплив масштабу часу на візуалізацію цифрового сигналу. При швидкості 9600 бод передача одного символу триває приблизно 1 мілісекунду. Спочатку, при встановленні масштабу розгортки (Time Div) у 500 мс, увесь екран охоплював період у 5 секунд. Через це мілісекундний сигнал передачі цілого байта відображався на екрані як один мікроскопічний сплеск напруги (спайк), оскільки ширина імпульсів була занадто малою для такого масштабу. Для коректного спостереження прямокутних імпульсів розгортку було зменшено до 100 мікросекунд на поділку. Саме при такому значенні стало можливим чітко побачити падіння напруги на старт-біті та послідовність інформаційних бітів.



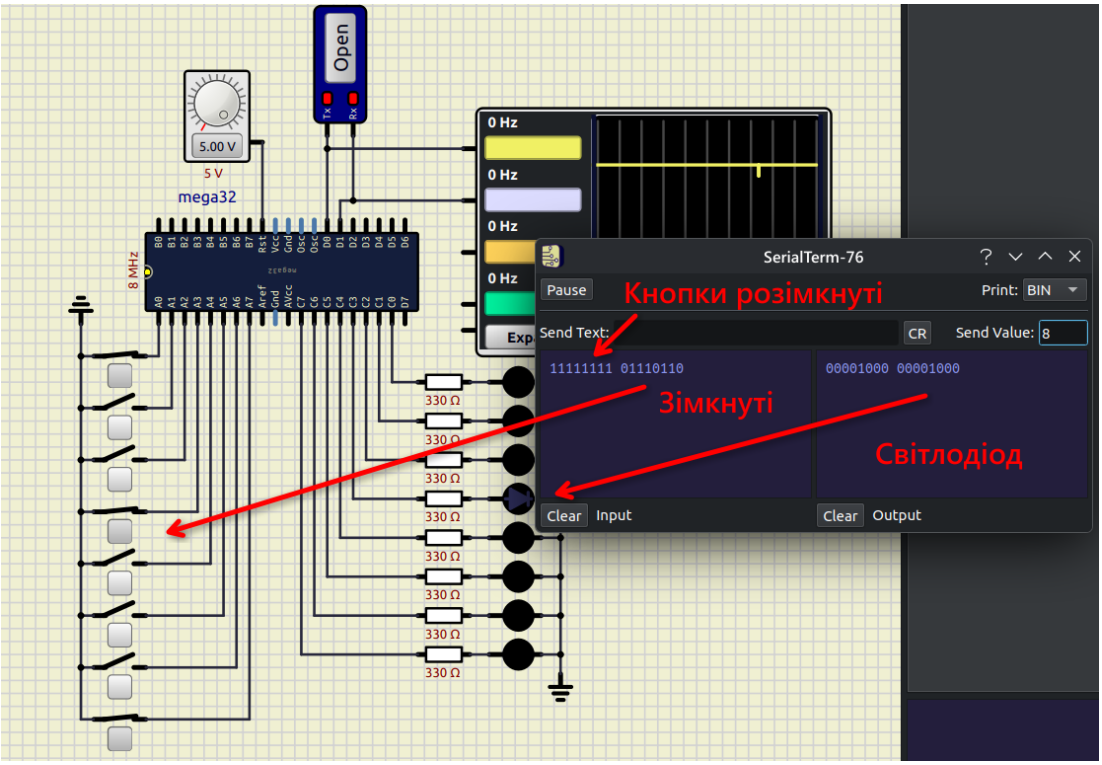
На отриманій осцилограмі чітко зафіксовано процес передачі даних послідовним каналом. На графіку можна спостерігати спади напруги від 5В до 0В, що відповідають стартовому біту кадру УАПП. Далі слідують інформаційні біти переданого символу (наприклад, десяткового значення 8) та стоп-біт, який повертає лінію у стан високого рівня. Одночасна поява імпульсів на обох каналах свідчить про успішний двосторонній обмін даними між мікроконтролером та віртуальним терміналом.

Для перевірки приймача мікроконтролера було здійснено передачу даних з віртуального термінала. При відправленні певного значення мікроконтролер успішно приймав цей байт і виводив його на порт C. Відповідно до двійкового коду отриманого числа засвічувалися відповідні світлодіоди. Наприклад, при відправленні десяткового числа 6, яке у двійковій системі має вигляд 00000110, на схемі засвічувався лише один світлодіод, підключений до відповідного піна порту C, що повністю підтверджує правильність роботи алгоритму прийому.



Робота передавача мікроконтролера перевірялася за допомогою перемикачів, підключених до порту A. Оскільки в програмі активовано внутрішні підтягуючі резистори, розімкнутий стан усіх перемикачів генерує на входах порту логічні одиниці, що відповідає двійковому числу 11111111 або десятковому 255. Мікроконтролер миттєво зчитував цей стан після розблокування циклу і відправляв його у термінал, де це значення відображалося в полі виводу. При замиканні певних перемикачів на відповідних пінах встановлювався логічний нуль, і мікроконтролер

успішно передавав нове значення, що засвідчує безпомилкову роботу підпрограми передачі.



Висновок. Під час виконання лабораторної роботи було успішно проведено моделювання модуля універсального асинхронного приймача-передавача (УАПП) мікроконтролера сім'ї AVR у середовищі SimulIDE. Практично досліджено процес конфігурації регістрів для встановлення швидкості передачі даних та формату кадру, а також вивчено апаратні особливості підключення ліній прийому-передачі (RX/TX). Налаштування стеку та обробка внутрішніх підтягуючих резисторів дозволили забезпечити стабільну роботу мікросхеми та коректне зчитування вхідних сигналів. Аналіз роботи пристрою за допомогою віртуального осцилографа наочно продемонстрував фізичні принципи формування часових інтервалів цифрового сигналу та дозволив засвоїти взаємозв'язок між швидкістю передачі (бодрейтом) і реальною тривалістю імпульсів напруги на лінії зв'язку.

Контрольні запитання.

1. **Що таке УСАПП (USART) та УАПП (UART)?**

   УСАПП (Універсальний Синхронний та Асинхронний Приймач-Передавач) – це апаратний модуль мікроконтролера, призначений для послідовного обміну даними з іншими пристроями як в асинхронному, так і в синхронному режимах. УАПП (Універсальний Асинхронний Приймач-Передавач) є спрощеним варіантом цього модуля, який підтримує виключно асинхрон-

ний режим зв'язку, де передача тактового сигналу між пристроями окремою лінією не передбачена.

2. **З яких трьох частин складається структура модуля УСАПП/УАПП? Які елементи містить кожна з них?**

   Структура модуля складається з генератора тактової частоти, передавача та приймача. Генератор містить логіку синхронізації та регістри налаштування швидкості передачі. Передавач складається з буферного регістра передачі, зсувного регістра передавача та блоку формування кадру. Приймач містить блок відновлення тактового сигналу та даних, зсувний регістр приймача, буферний регістр прийому та блок контролю парності.

3. **Які регістри використовуються для керування модулями УАПП та УСАПП?**

   Основними керуючими регістрами є регістр даних UDR (фізично розділений на окремі регістри для читання та запису за однією адресою), регістри керування та статусу UCSRA, UCSRB та UCSRC, а також регістри налаштування швидкості передачі даних UBRRH та UBRRL.

4. **Чим в асинхронному режимі задається швидкість прийому та передачі даних?**

   В асинхронному режимі швидкість задається внутрішнім генератором швидкості передачі (Baud Rate Generator). Конкретне значення швидкості визначається числом, яке програміст записує у 16-розрядний регістр UBRR (складається з UBRRH та UBRRL), і воно математично залежить від частоти системного тактового генератора мікроконтролера.

5. **Поясніть структурну схему блока синхронізації модуля УСАПП для мікроконтролера Mega 128.**

   Блок синхронізації отримує тактовий сигнал від системного осцилятора мікроконтролера і ділить його частоту за допомогою дільника з програмованим коефіцієнтом ділення, який задається регістром UBRR. Отриманий тактовий сигнал використовується для керування зсувними регістрами приймача та передавача. Для асинхронного режиму також застосовується додаткове ділення частоти на 16 або на 8 для забезпечення надійної вибірки даних на стороні приймача.

6. **Які режими синхронізації підтримує УСАПП?**

   Модуль УСАПП підтримує три основні режими синхронізації. Це асинхронний нормальний режим, асинхронний режим з подвоєнням швидкості передачі, а також синхронний режим ведучого або веденого пристрою, у якому

тактовий сигнал обов'язково передається окремою фізичною лінією XCK.

7. **Як визначається швидкість обміну в модулі УСАПП?**
   Швидкість обміну обчислюється за формулами, які пов'язують бажану швидкість у бодах, частоту системного тактового генератора (f_{osc}) та значення регістра UBRR. Наприклад, у нормальному асинхронному режимі значення регістра розраховується як $UBRR = \frac{f_{osc}}{16 \cdot BAUD} - 1$.
8. **Як визначається розмір слова даних у модулях УСАПП?**
   Розмір слова даних задається комбінацією бітів UCSZ2, UCSZ1 та UCSZ0, які знаходяться в регістрах керування UCSRB та UCSRC. Мікроконтролери AVR дозволяють програмно налаштувати розмір інформаційної частини кадру на 5, 6, 7, 8 або 9 біт даних.
9. **Як працює схема контролю парності?**
   Схема контролю парності автоматично підраховує кількість логічних одиниць у слові даних. Залежно від налаштувань, генератор формує додатковий біт парності або непарності, який додається до кадру при передачі. На приймальній стороні апаратний блок перевіряє отриманий кадр разом із бітом парності, і у разі математичної невідповідності встановлює прапорець помилки UPE у регістрі статусу UCSRA.
10. **Як проходить передача даних в модулях УСАПП?**
    Процес передачі ініціюється записом байта даних у регістр UDR. Далі ці дані автоматично переміщуються у зсувний регістр передавача, щойно він звільняється. Апаратний блок передавача додає старт-біт, розрахований біт парності та стоп-біти, після чого послідовно видає сформований кадр на лінію TXD, починаючи з молодшого значущого біта.
11. **Як проходить прийом даних в модулях УСАПП?**
    Приймач постійно відстежує лінію RXD на наявність спаду напруги, що сигналізує про старт-біт. Виявивши його, блок відновлення даних здійснює багаторазову вибірку стану лінії для кожного наступного біта. Отримані біти послідовно завантажуються у зсувний регістр приймача. Коли прийом усього кадру завершено, дані переміщуються у приймальний буфер UDR, а в регістрі статусу встановлюється прапорець завершення прийому RXC.
12. **Що таке режим мультиконтролерного обміну?**
    Це спеціальний режим роботи УСАПП, який використовується для організації мережі з кількома мікроконтролерами на одній спільній шині. Він дозволяє веденим пристроям ігнорувати кадри з даними, які призначені не їм, а апаратно реагувати лише на адресні кадри, що суттєво зменшує наванта-

ження на процесорний час ведених вузлів мережі.

13. **Опишіть послідовність дій для здійснення обміну даними у мікроконтролерній мережі.**

    Ведучий мікроконтролер формує адресний кадр, вказуючи адресу цільового веденого пристрою, і передає його з встановленим 9-м бітом як ідентифікатором адреси. Усі ведені пристрої, у яких увімкнено режим мультиконтролерного обміну, приймають цей кадр і порівнюють адресу зі своєю. Мікроконтролер, адреса якого збіглася, скидає спеціальний біт і починає приймати наступні кадри даних звичайним чином, тоді як інші пристрої продовжують ігнорувати обмін.

14. **Як визначити швидкість передачі даних для асинхронного звичайного режиму?**

    Швидкість передачі (BAUD) в асинхронному нормальному режимі розраховується на основі системної тактової частоти (f_{osc}) та поточного значення регістра UBRR за математичною формулою: $BAUD = \frac{f_{osc}}{16 \cdot (UBRR+1)}$.

15. **Як визначити тривалість передачі кадру для асинхронного прискореного режиму?**

    У прискореному режимі апаратний дільник тактової частоти зменшується з 16 до 8, що робить швидкість удвічі вищою при тому самому значенні UBRR. Тривалість передачі всього кадру визначається як загальна кількість бітів у ньому (старт-біт, дані, парність, стоп-біти), поділена на фактичну швидкість передачі у бодах.

16. **Як в моделі підключено кнопки, які задають байт для передачі?**

    У розробленій моделі використовується блок фіксованих перемикачів, кожен з яких підключений безпосередньо до відповідного піна порту A мікроконтролера. Інший контакт кожного перемикача з'єднаний із загальною лінією землі.

17. **Як в моделі підключено світлодіоди, які відображають передану в моделі інформацію?**

    Світлодіоди підключені до пінів порту C. Аноди світлодіодів з'єднані з виводами мікроконтролера через струмообмежувальні резистори номіналом 330 Ом, а катоди всіх світлодіодів об'єднані та підключені до загальної лінії землі.

18. **Як в моделі визначається тривалість переданого біта?**

    Тривалість переданого біта є обернено пропорційною до заданої швидкості передачі даних. Оскільки в моделі та коді встановлено швидкість 9600 бод,

фізична тривалість одного біта розраховується як $\frac{1}{9600}$ секунди, що становить приблизно 104 мікросекунди.

19. **Опишіть формат та призначення розрядів регістра UCSRC.**

Регістр UCSRC використовується для налаштування формату кадру. Біт URSEL визначає вибір доступу між регістрами (має бути 1 для запису в UCSRC), біт UMSEL задає синхронний чи асинхронний режим роботи. Біти UPM1 та UPM0 налаштовують перевірку парності, біт USBS задає кількість стоп-бітів, а біти UCSZ1 та UCSZ0 визначають кількість бітів даних у кадрі. У розробленій програмі цей регістр налаштовано на 8 біт даних, 1 стоп-біт і відсутність перевірки парності.

20. **Як в програмі відбувається налаштування порту A на передачу?**

Налаштування порту A для зчитування кнопок виконується у два кроки. Спочатку в регістр напрямку даних DDRA записується значення 0x00, що налаштовує всі піни порту як входи. Далі у регістр даних PORTA записується значення 0xFF, що програмно активує внутрішні підтягуючі резистори для всіх пінів цього порту, забезпечуючи надійне зчитування стану розімкнутих перемикачів як логічної одиниці.

21. **Як в програмі визначається час завершення передачі та прийому чергового байта?**

Час завершення операцій визначається шляхом програмного опитування відповідних прапорців у регістрі статусу UCSRA. Підпрограма передачі в циклі перевіряє стан прапорця UDRE (регістр даних вільний) і чекає його встановлення перед записом нового байта. Підпрограма прийому в циклі чекає встановлення прапорця RXC (прийом завершено), після чого зчитує отриманий байт з регістра UDR.

22. **Дайте характеристику ASCII-коду.**

ASCII – це стандартизована таблиця кодування, у якій кожному друкованому символу (літерам, цифрам, розділовим знакам) та спеціальним керуючим символам відповідає унікальний числовий код. При відправленні тексту через віртуальний термінал символи автоматично перетворюються на відповідні 8-бітні двійкові значення ASCII-коду.

23. **Поясніть як в моделі підключено світлодіоди та в якому випадку вони будуть світитися?**

Аноди світлодіодів підключені до пінів порту C через резистори 330 Ом, а катоди заземлені. Оскільки в програмі порт C налаштований на вивід, світлодіоди світитимуться лише тоді, коли мікроконтролер виставить на відпо-

відних пінах порту логічну одиницю. Це відбувається, коли у прийнятому з термінала байті відповідний біт має значення одиниці.

24. **Як працюють схеми відновлення тактового сигналу і даних при асинхронному прийомі?**

Оскільки тактовий сигнал не передається лінією, приймач повинен самостійно синхронізуватися з передавачем. Схема відновлення використовує внутрішній генератор, який працює на частоті, що в 16 або 8 разів перевищує задану швидкість передачі. При виявленні спаду напруги на старт-біті схема здійснює багаторазову вибірку стану лінії в центрі кожного наступного бітового інтервалу, використовуючи мажоритарне голосування для захисту від завад і прийняття остаточного рішення про стан біта.

Лістинг.

firmware.asm

```
.nolist
.include "m32def.inc"
.list

.equ fck = 8000000
.equ BAUD = 9600
.equ UBRR_value = (fck/(BAUD*16))-1

.cseg
.org 0

Start:
; Initialize Stack Pointer
ldi R16, high(RAMEND)
out SPH, R16
ldi R16, low(RAMEND)
out SPL, R16

rcall init_USART

; Configure PORTC as output for LEDs
ldi R16, 0xFF
out DDRC, R16

Main_Loop:
; Receive data from terminal
rcall USART_receive
; Output received data to LEDs
out PORTC, R16

; Configure PORTA as input for switches
ldi R16, 0x00
out DDRA, R16

; Enable internal pull-up resistors for PORTA
ldi R16, 0xFF
out PORTA, R16

; Read switches state
in R16, PINA
; Send data to terminal
rcall USART_send

rjmp Main_Loop

init_USART:
; Set baud rate
ldi R16, high(UBRR_value)
out UBRRH, R16
ldi R16, low(UBRR_value)
out UBRRL, R16

; Enable receiver and transmitter
ldi R16, (1<<RXEN)|(1<<TXEN)|(0<<RXCIE)|(0<<TXCIE)|(0<<UDRIE)
out UCSRB, R16

; Set frame format: 8 data bits, no parity, 1 stop bit
ldi R16, (1<<URSEL)|(1<<UCSZ1)|(1<<UCSZ0)
```

```
out UCSRC, R16
ret

USART_send:
; Wait for empty transmit buffer
sending:
sbis UCSRA, UDRE
rjmp sending
; Put data into buffer, sends the data
out UDR, R16
ret

USART_receive:
; Wait for data to be received
receiving:
sbis UCSRA, RXC
rjmp receiving
; Get and return received data from buffer
in R16, UDR
ret
```